

Final Conclusion: Expression Evaluation Methods

Result Verification: Both stack-based postfix evaluation and expression tree evaluation consistently yield the exact same result, **11**, for the given postfix expression.

Stack-Based Postfix Evaluation

Stack-based postfix evaluation is remarkably simple and computationally efficient. It is purpose-built for directly evaluating a postfix expression without requiring complex intermediate data structures.

Complexity: $O(n)$ time complexity with minimal auxiliary space usage.

Expression Tree Evaluation

The Expression Tree shares the same $O(n)$ time complexity, but introduces powerful structural advantages by representing operands and operators in a clear hierarchical form.

Capabilities: Enables easy generation of preorder, inorder, and postorder traversals, facilitating deeper expression analysis or algorithmic modification.

Methodology Comparison Summary

Feature	Stack-Based Postfix	Expression Tree
Primary Objective	Direct evaluation	Structural representation & traversal
Time Complexity	$O(n)$	$O(n)$
Structural Insight	None (linear processing)	Hierarchical tree structure
Traversals Supported	Postfix scan only	Preorder, Inorder, Postorder

Final Verdict: Postfix evaluation is exceptionally useful for direct, streamlined evaluation tasks. Conversely, an Expression Tree is indispensable when the underlying hierarchical structure of the expression is required for analysis, transformation, or translation.